```
Output:
Fatal error: Uncaught TypeError: testArgs(): Argument #6 must be of type
string, stdClass given
```

Now if we run this code, we still get an error, albeit a different one (a TypeError rather than a string conversion failure). PHP now fails to call `testArgs()` at all, rather than getting most of the way through the `foreach` loop before dying. The real benefit here is that the declaration is a statement of the function's behavior; it tells other programmers (including future you) how to use `testArgs()` correctly. In addition, a good IDE will highlight the type mismatch problem before the code ever runs.

It is also possible to mix conventional parameters with variadic (with the caveat that only the final one can be variadic):

```
function testArgs(string $first, string ...$rest): void
{
    print $first . ' ';

    foreach ($rest as $str) {
        print $str . ' ';
    }
}

testArgs('I', 'hope', 'this', 'makes', 'sense', 'too');
```

```
Output:
I hope this makes sense too
```

It is also perfectly fine to unpack arrays when using them as function arguments. If our six strings were collected into an array, we would make the call like this:

```
$strings = ['I', 'hope', 'this', 'makes', 'sense', 'too'];
testArgs(...$strings);
```

When the data arrives into the function, it will have been split into individual arguments. Prior to array unpacking being added to PHP (in version 5.6), there was another way to spread array elements when calling functions, using the built-in `call_user_func_array()`:

```
$strings = ['I', 'hope', 'this', 'makes', 'sense', 'too'];
call_user_func_array('testArgs', $strings);
```